

MBA BRAIN AGENT

Codigo Fonte

Sistema completo de DMs automaticos via HTTP com cookies
Instagram + Facebook + TikTok

7 ficheiros | ~1.900 linhas | Next.js / TypeScript

src/middleware.ts

Autenticacao: Bearer MBA2026 ou header x-mba-session | 20 linhas

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

const PUBLIC_PATHS = ['/api/auth/login', '/api/webhook', '/api/auth/check'];

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const isPublic = PUBLIC_PATHS.some(p => pathname.startsWith(p));
  if (isPublic) return NextResponse.next();
  if (!pathname.startsWith('/api/')) return NextResponse.next();
  const authHeader = request.headers.get('authorization');
  const sessionHeader = request.headers.get('x-mba-session');
  const validCode = process.env.ACCESS_CODE || 'MBA2026';
  if (authHeader === 'Bearer ' + validCode) return NextResponse.next();
  if (sessionHeader && sessionHeader.length > 0) return NextResponse.next();
  return NextResponse.json({ error: 'Nao autorizado' }, { status: 401 });
}

export const config = { matcher: ['/api/:path*'] };
```

src/app/api/cookies/route.ts

Stub vazio - 501 nao funcional em serverless | 9 linhas

```
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({ cookies: [] });
}

export async function PATCH() {
  return NextResponse.json({ error: 'Gestao de cookies via ficheiro nao disponivel no modo serverless' }, { status: 501 });
}
```

src/app/api/send-message/route.ts

PRINCIPAL - valida cookies, busca user ID, envia DMs por HTTP para IG, FB, TT | 455 linhas

```

import { NextResponse } from 'next/server';

export var maxDuration = 60;
export var MAX_PER_DAY = 30;

async function igGetUserId(sessionid: string, csrftoken: string, username: string): Promise<string | null> {
  try {
    var res = await fetch('https://www.instagram.com/api/v1/users/web_profile_info/?username=' + encodeURIComponent(username), {
      headers: {
        'cookie': 'sessionid=' + sessionid + '; csrftoken=' + csrftoken,
        'x-csrftoken': csrftoken,
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36',
        'x-ig-app-id': '936619743392459'
      },
      signal: AbortSignal.timeout(15000)
    });
    if (!res.ok) return null;
    var data = await res.json();
    return data.data?.user?.pk || data.data?.user?.id || null;
  } catch (e) { return null; }
}

async function igSendDM(sessionid: string, csrftoken: string, recipientId: string, message: string): Promise<{ success: boolean; error?: string }> {
  try {
    var threadRes = await fetch('https://www.instagram.com/api/v1/direct_v2/create_thread/', {
      method: 'POST',
      headers: {
        'cookie': 'sessionid=' + sessionid + '; csrftoken=' + csrftoken,
        'x-csrftoken': csrftoken,
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36',
        'x-ig-app-id': '936619743392459',
        'content-type': 'application/x-www-form-urlencoded'
      },
      body: 'recipient_users=' + JSON.stringify([recipientId]) + '&client_context=' + Date.now() + '&device_id=' + crypto.randomUUID() || (Math.random().toString(36) + Date.now().toString(36)),
      signal: AbortSignal.timeout(15000)
    });
    var threadData = await threadRes.json();
    var threadId = threadData.thread_id || threadData.thread?.thread_id;
    if (!threadId) {
      var inboxRes = await fetch('https://www.instagram.com/api/v1/direct_v2/inbox/?user_id=' + recipientId, {
        headers: {
          'cookie': 'sessionid=' + sessionid + '; csrftoken=' + csrftoken,
          'x-csrftoken': csrftoken, 'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
          'x-ig-app-id': '936619743392459'
        },
        signal: AbortSignal.timeout(10000)
      });
      if (inboxRes.ok) {
        var inboxData = await inboxRes.json();
        if (inboxData.inbox && inboxData.inbox.threads) {
          for (var t of inboxData.inbox.threads) {
            if (t.users && t.users.length > 0 && String(t.users[0].pk) === String(recipientId)) {
              threadId = t.thread_id; break;
            }
          }
        }
      }
    }
    if (!threadId) return { success: false, error: 'Nao conseguiu criar/encontrar thread de DM' };
    var msgRes = await fetch('https://www.instagram.com/api/v1/direct_v2/threads/' + threadId + '/items/', {
      method: 'POST',
      headers: {
        'cookie': 'sessionid=' + sessionid + '; csrftoken=' + csrftoken,
        'x-csrftoken': csrftoken, 'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
        'x-ig-app-id': '936619743392459', 'content-type': 'application/x-www-form-urlencoded'
      },
      body: 'text=' + encodeURIComponent(message) + '&client_context=' + Date.now(),
      signal: AbortSignal.timeout(15000)
    });
    if (msgRes.ok) return { success: true };
    return { success: false, error: 'Erro ao enviar mensagem (HTTP ' + msgRes.status + ')' };
  } catch (e: any) { return { success: false, error: e.message || 'timeout' }; }
}

async function igValidateCookies(sessionid: string, csrftoken: string) {
  try {
    var res = await fetch('https://www.instagram.com/api/v1/accounts/current_user/?fields=username', {
      headers: {
        'cookie': 'sessionid=' + sessionid + '; csrftoken=' + csrftoken, 'x-csrftoken': csrftoken,
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
        'x-ig-app-id': '936619743392459'
      },
      signal: AbortSignal.timeout(10000)
    });
    if (res.ok) { var data = await res.json(); return { valid: true, username: data.user?.username || data.username || 'OK' }; }
    if (res.status === 401 || res.status === 403) return { valid: false, error: 'Cookies expirados.' };
    return { valid: false, error: 'Erro HTTP ' + res.status };
  } catch (e: any) { return { valid: false, error: e.message || 'Erro de conexao' }; }
}

// Mensagens recebidas no inbox do IG e TT via cookies | 210 linhas

async function fbValidateCookies(fbCookie: string, fbDtsg: string) {
  try {
    var res = await fetch('https://www.facebook.com/api/graphql/', {
      method: 'POST',
      headers: { 'cookie': fbCookie, 'content-type': 'application/x-www-form-urlencoded',
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36' },
      body: 'fb_dtsg=' + encodeURIComponent(fbDtsg) + '&doc_id=4364992440242396&variables=%7B%22scale%22%3A3%7D',
      signal: AbortSignal.timeout(10000)
    });
  }
}

```

src/app/api/inbox/route.ts

```
var allMessages = []; var logs = [];
src/app/api/scrape/route.ts
```

Scraping via Apify + Meta Graph API + sistema de scoring | 602 linhas


```

const META_TOKEN = process.env.META_ACCESS_TOKEN || '';
const APIFY_KEY = process.env.APIFY_API_KEY || '';

interface ScrapedProfile {
  platform: string; username: string; displayName: string | null; followers: number; following: number;
  postsCount: number; monthsActive: number; isRegular: boolean; isVerified: boolean; bio: string | null;
  profileUrl: string; avatarUrl: string | null; category: string | null; location: string | null;
  isBusiness: boolean; externalId: string;
}

export async function scrapeInstagram(query: string, filters: any): Promise<ScrapedProfile[]> {
  const profiles: ScrapedProfile[] = [];
  try {
    const response = await fetch('https://api.apify.com/v2/acts/apify/instagram-scraper/runs', {
      method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + APIFY_KEY },
      body: JSON.stringify({ searchQueries: [query || 'Luanda Angola'], resultsType: 'profiles', resultsLimit: filters.targetCount * 2 })
    });
    if (response.ok) {
      const run = await response.json(); const runId = run.data?.id || run.id;
      if (runId) { await new Promise(r => setTimeout(r, 5000));
        const resultsRes = await fetch('https://api.apify.com/v2/actor-runs/' + runId + '/dataset/items?limit=' + (filters.targetCount * 2), { headers: { 'Authorization': 'Bearer ' + APIFY_KEY } });
        if (resultsRes.ok) { const items = await resultsRes.json();
          for (const item of items) { const p = mapInstagramProfile(item); if (p && passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } }
      }
    }
  } catch (e) { console.error('[IG Apify] Error:', e); }
  if (profiles.length < filters.targetCount) {
    try {
      const searchUrl = 'https://graph.facebook.com/v21.0/ig_business_discovery/search?user_id=17841400191564710&q=' + encodeURIComponent(query || 'Luanda Angola') + '&fields=business_discovery.username,business_discovery.followers_count,business_discovery.media_count,business_discovery.biography&access_token=' + META_TOKEN;
      const res = await fetch(searchUrl);
      if (res.ok) { const data = await res.json(); const bd = data.business_discovery;
        if (bd?.username) { const p: ScrapedProfile = { platform: 'instagram', username: '@' + bd.username, displayName: bd.name, followers: bd.followers_count || 0, following: 0, postsCount: bd.media_count || 0, monthsActive: 12, isRegular: true, isVerified: false, bio: bd.biography || null, profileUrl: 'https://instagram.com/' + bd.username, avatarUrl: null, category: null, location: 'Angola', isBusiness: true, externalId: bd.username };
          if (passesFilters(p, filters)) profiles.push(p); } }
      } catch (e) { console.error('[IG Meta API] Error:', e); }
    }
    return profiles.slice(0, filters.targetCount);
  }
}

export async function scrapeFacebook(query: string, filters: any): Promise<ScrapedProfile[]> {
  const profiles: ScrapedProfile[] = [];
  try {
    const url = 'https://graph.facebook.com/v21.0/pages/search?q=' + encodeURIComponent(query || 'Luanda Angola') + '&fields=id,name,username,fan_count,category,about,created_time&limit=100&access_token=' + META_TOKEN;
    const res = await fetch(url);
    if (res.ok) { const data = await res.json();
      for (const page of (data.data || [])) {
        const monthsActive = Math.max(1, Math.floor((Date.now() - new Date(page.created_time || Date.now()).getTime()) / 259200000));
        const p: ScrapedProfile = { platform: 'facebook', username: page.username ? '@' + page.username : 'page-' + page.id, displayName: page.name, followers: page.fan_count || 0, following: 0, postsCount: 0, monthsActive, isRegular: true, isVerified: false, bio: page.about || null, profileUrl: 'https://facebook.com/' + (page.username || page.id), avatarUrl: null, category: page.category || null, location: 'Angola', isBusiness: true, externalId: page.id };
        if (passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } }
      } catch (e) { console.error('[FB Meta API] Error:', e); }
    return profiles.slice(0, filters.targetCount);
  }
}

export async function scrapeTikTok(query: string, filters: any): Promise<ScrapedProfile[]> {
  const profiles: ScrapedProfile[] = [];
  try {
    const response = await fetch('https://api.apify.com/v2/acts/apify/tiktok-scraper/runs', {
      method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + APIFY_KEY },
      body: JSON.stringify({ hashtags: [query || 'Luanda'], resultsPerPage: filters.targetCount * 2, shouldDownloadVideos: false })
    });
    if (response.ok) { const run = await response.json(); const runId = run.data?.id || run.id;
      if (runId) { await new Promise(r => setTimeout(r, 8000));
        const resultsRes = await fetch('https://api.apify.com/v2/actor-runs/' + runId + '/dataset/items?limit=' + (filters.targetCount * 2), { headers: { 'Authorization': 'Bearer ' + APIFY_KEY } });
        if (resultsRes.ok) { const items = await resultsRes.json(); const seen = new Set<string>();
          for (const item of items) { const author = item.author?.uniqueId || item.author?.id;
            if (author && !seen.has(author)) { seen.add(author);
              const p: ScrapedProfile = { platform: 'tiktok', username: '@' + author, displayName: item.author?.nickname || author, followers: item.author?.fans || 0, following: item.author?.following || 0, postsCount: item.author?.videoCount || 0, monthsActive: 12, isRegular: true, isVerified: item.author?.verified || false, bio: item.author?.signature || null, profileUrl: 'https://tiktok.com/@' + author, avatarUrl: null, category: null, location: null, isBusiness: false, externalId: item.author?.id || author };
              if (passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } } }
      } catch (e) { console.error('[TT Apify] Error:', e); }
    return profiles.slice(0, filters.targetCount);
  }
}

```

```

export async function scrapeLinkedIn(query: string, filters: any): Promise<ScrapedProfile[]> {
  const profiles: ScrapedProfile[] = [];
  try {
    const response = await fetch('https://api.apify.com/v2/acts/apify/linkedin-profile-scraper/runs', {
      method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + APIFY_KEY },
      body: JSON.stringify({ searchQueries: [query || 'Luanda Angola'], maxResults: filters.targetCount * 2 })
    });
    if (response.ok) { const run = await response.json(); const runId = run.data?.id || run.id;
      if (runId) { await new Promise(r => setTimeout(r, 8000));
        const resultsRes = await fetch('https://api.apify.com/v2/actor-runs/' + runId + '/dataset/items?limit=' + (filters.targetCount * 2), { headers: { 'Authorization': 'Bearer ' + APIFY_KEY } });
        if (resultsRes.ok) { const items = await resultsRes.json();
          for (const item of items) { const username = item.username || ((item.firstName || '').toLowerCase() + '-' + (item.lastName || ''));
            const p: ScrapedProfile = { platform: 'linkedin', username: username, displayName: item.firstName || item.lastName || item.fullName || '', followers: item.followersCount || 0, following: item.followingCount || 0, postsCount: item.postsCount || 0, monthsActive: 12, isRegular: true, isVerified: item.isVerified || false, bio: item.bio || null, profileUrl: 'https://linkedin.com/in/' + username, avatarUrl: item.avatarUrl || null, category: item.category || null, location: item.location || null, isBusiness: item.isBusiness || false, externalId: item.externalId || null };
            if (passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } }
      } catch (e) { console.error('[LI Apify] Error:', e); }
    return profiles.slice(0, filters.targetCount);
  }
}

```

src/app/setup-cookies/page.tsx

UI dark mobile-friendly para colar/validar/guardar cookies no localStorage | 319 linhas

```

const response = await fetch('https://api.apify.com/v2/acts/apify/linkedin-profile-scraper/runs', {
  method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + APIFY_KEY },
  body: JSON.stringify({ searchQueries: [query || 'Luanda Angola'], maxResults: filters.targetCount * 2 })
});
if (response.ok) { const run = await response.json(); const runId = run.data?.id || run.id;
  if (runId) { await new Promise(r => setTimeout(r, 8000));
    const resultsRes = await fetch('https://api.apify.com/v2/actor-runs/' + runId + '/dataset/items?limit=' + (filters.targetCount * 2), { headers: { 'Authorization': 'Bearer ' + APIFY_KEY } });
    if (resultsRes.ok) { const items = await resultsRes.json();
      for (const item of items) { const username = item.username || ((item.firstName || '').toLowerCase() + '-' + (item.lastName || ''));
        const p: ScrapedProfile = { platform: 'linkedin', username: username, displayName: item.firstName || item.lastName || item.fullName || '', followers: item.followersCount || 0, following: item.followingCount || 0, postsCount: item.postsCount || 0, monthsActive: 12, isRegular: true, isVerified: item.isVerified || false, bio: item.bio || null, profileUrl: 'https://linkedin.com/in/' + username, avatarUrl: item.avatarUrl || null, category: item.category || null, location: item.location || null, isBusiness: item.isBusiness || false, externalId: item.externalId || null };
        if (passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } }
    } catch (e) { console.error('[LI Apify] Error:', e); }
  }
  return profiles.slice(0, filters.targetCount);
}

```

```

useEffect(() => {
  const p: ScrapedProfile = { platform: 'linkedin', username: '@' + username, displayName: ((item.firstName || '') + ' ' + (item.lastName || '')).trim(), followers: item.connectionsCount || 0, following: 0, postsCount: item.postsCount || 0, monthsActive: 12, isVerified: item.isVerified || false, bio: item.summary || item.headline || null, profileUrl: item.url || 'https://linkedin.com/in/' + username, avatarUrl: null, category: item.title || null, location: item.location || null, isBusiness: false, externalId: item.id || item.id };
  if (passesFilters(p, filters)) profiles.push(p); if (profiles.length >= filters.targetCount) break; } } }
export default function SetupCookies() {
  const [igSessionid, setIgSessionid] = useState('');
  const [igCsrf, setIgCsrf] = useState('');
  const [fbCookie, setFbCookie] = useState('');
  const [fbDtsG, setFbDtsG] = useState('');
  function passesFilters(p: ScrapedProfile, f: any): boolean {
    const { igSessionid, setIgSessionid } = useState('');
    const { igCsrf, setIgCsrf } = useState('');
    const { fbCookie, setFbCookie } = useState('');
    const { fbDtsG, setFbDtsG } = useState('');
    if (p.monthsActive < f.minMonthsActive) return false;
    if (p.followers < f.minFollowers) return false;
    if (p.postsCount < f.minPostsCount) return false;
    if (p.isVerified && !f.isVerified) return false;
    if (p.isBusiness && !f.isBusiness) return false;
    if (p.category && !f.category) return false;
    if (p.location && !f.location) return false;
    if (p.externalId && !f.externalId) return false;
    return true;
  }
  useEffect(() => {
    const d = JSON.parse(localStorage.getItem('mba_cookies') || '{}');
    if (d.igSessionid) setIgSessionid(d.igSessionid); if (d.igCsrf) setIgCsrf(d.igCsrf);
    if (d.fbCookie) setFbCookie(d.fbCookie); if (d.fbDtsG) setFbDtsG(d.fbDtsG);
    return { platform: d.platform, username: d.username, followers: item.followersCount || 0, postsCount: item.postsCount || 0, monthsActive: 12, isRegular: (item.postsCount || 0) > 0, isVerified: item.isVerified || false, bio: item.biography || null, profileUrl: 'https://instagram.com/' + item.username, avatarUrl: item.profilePicUrl || null, category: null, location: item.locationName || null, isBusiness: item.isBusinessAccount || false, externalId: item.id || item.username };
    function getHeaders() {
      const session = localStorage.getItem('mba_session') || '';
      return { 'Authorization': 'Bearer MBA2026' + session };
    }
    const cats: Record<string, string[]> = { 'Restauracao': ['restaurante', 'comida', 'food', 'gastronomia'], 'Tecnologia': ['tech', 'software', 'app', 'digital', 'startup'], 'Marketing': ['marketing', 'publicidade', 'branding'], 'Moda': ['moda', 'fashion', 'estilo', 'roupa'], 'Saude': ['saude', 'health', 'clinica', 'fitness'], 'Educacao': ['educacao', 'escola', 'formacao', 'curso'], 'Imobiliario': ['imoveis', 'imobiliario', 'construcao', 'eventos', 'eventb', 'feira', 'wedding', 'casamento'], 'Consultoria': ['consultoria', 'consulting', 'coaching'], 'Lazer': ['lazer', 'viagem', 'eventos', 'eventb', 'feira', 'wedding', 'casamento'], 'Consultoria': ['consultoria', 'consulting', 'coaching'] };
    for (const [cat, keywords] of Object.entries(cats)) { if (keywords.some(k => k.includes(k))) return cat; }
    return { platform: 'facebook', body: fbCookie, body: fbDtsG, body: fbDtsG };
  }
  if (platform === 'tiktok') { body.ttSessionid = ttSessionid.trim(); body.ttCsrf = ttCsrf.trim(); }
  try { const res = await fetch(API, { method: 'POST', headers: getHeaders(), body: JSON.stringify(body) });
    const data = await res.json();
    export function calculateScore(profile: ScrapedProfile): number {
      let score = 0;
      score += Math.min(40, Math.round((profile.followers / 50000) * 40));
      score += Math.min(30, Math.round((profile.postsCount / 1000) * 30));
      score += Math.min(20, Math.round((profile.monthsActive / 48) * 20));
      if (!profile.isVerified && !profile.isBusiness) score += 10;
      return Math.min(100, score);
    }
    function saveCookies() {
      const data: any = {};
      if (igSessionid.trim()) data.igSessionid = igSessionid.trim(); if (igCsrf.trim()) data.igCsrf = igCsrf.trim();
      if (fbCookie.trim()) data.fbCookie = fbCookie.trim(); if (fbDtsG.trim()) data.fbDtsG = fbDtsG.trim();
      if (ttSessionid.trim()) data.ttSessionid = ttSessionid.trim(); if (ttCsrf.trim()) data.ttCsrf = ttCsrf.trim();
      localStorage.setItem('mba_cookies', JSON.stringify(data)); setSaved(true); setTimeout(() => setSaved(false), 3000);
    }
  }
}

function testDM(platform: string) {
  const usernames: Record<string, string> = { instagram: 'meta', facebook: 'zuck', tiktok: 'tiktok' };
  const testUser = usernames[platform] || 'test';
  if (confirm('Enviar DM de teste para @' + testUser + '?')) {
    const cookies = JSON.parse(localStorage.getItem('mba_cookies') || '{}');
    let body: any = { platform, username: testUser, message: 'Teste DM - ignorar', sentToday: 0 };
    if (platform === 'instagram') { body.igSessionid = cookies.igSessionid; body.igCsrf = cookies.igCsrf; }
    if (platform === 'facebook') { body.fbCookie = cookies.fbCookie; body.fbDtsG = cookies.fbDtsG; }
    if (platform === 'tiktok') { body.ttSessionid = cookies.ttSessionid; body.ttCsrf = cookies.ttCsrf; }
    fetch(API, { method: 'POST', headers: getHeaders(), body: JSON.stringify(body) })
      .then(r => r.json())
      .then(d => alert('Resultado: ' + (d.deliveryMsg || JSON.stringify(d))))
      .catch(e => alert('Erro: ' + e.message));
  }
}

const platformConfig = [
  { id: 'instagram', name: 'Instagram', icon: 'IG', color: '#E1306C', fields: [
    { label: 'sessionid', value: igSessionid, set: setIgSessionid, placeholder: 'Copiado do browser' },
    { label: 'csrfToken', value: igCsrf, set: setIgCsrf, placeholder: 'Copiado do browser' } ] },
  { id: 'facebook', name: 'Facebook', icon: 'FB', color: '#1877F2', fields: [
    { label: 'cookie (tudo)', value: fbCookie, set: setFbCookie, placeholder: 'Cole TODOS os cookies', big: true },
    { label: 'fb_dtsG', value: fbDtsG, set: setFbDtsG, placeholder: 'Valor de fb_dtsG' } ] },
  { id: 'tiktok', name: 'TikTok', icon: 'TT', color: '#25F4EE', fields: [
    { label: 'sessionid', value: ttSessionid, set: setTtSessionid, placeholder: 'Copiado do browser' },
    { label: 'tt_csrf_token', value: ttCsrf, set: setTtCsrf, placeholder: 'Copiado do browser' } ] }
];

return (
  <div style={{ minHeight: '100vh', background: '#0a0a0a', color: '#e0e0e0', fontFamily: 'system-ui, sans-serif', padding: '16px 0 16px 0', paddingBottom: '100px' }}>
    <div style={{ textAlign: 'center', padding: '24px 0 20px', borderBottom: '1px solid #222', marginBottom: '24px' }}>
      <h1 style={{ fontSize: '24px', fontWeight: 700, color: 'fff', margin: '0 8px' }}>Configurar Cookies</h1>
      <p style={{ fontSize: '14px', color: '#888', margin: 0 }}>Cola os cookies de cada plataforma para ativar os DMs</p>
    </div>
    {platformConfig.map(platform => {
      const status = statuses[platform.id];
      return (
        <div key={platform.id} style={{ background: '#111', border: '1px solid #222', borderRadius: '16px', padding: '20px', margin: '16px 0 16px 0' }}>
          <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', margin: '16px 0 16px 0' }}>
            <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
              <div style={{ width: '44px', height: '44px', borderRadius: '12px', background: platform.color, display: 'flex', alignItems: 'center', justifyContent: 'center', color: platform.id === 'tiktok' ? '#000' : 'fff', fontWeight: 700, fontSize: '16px' }}>{platform.icon}</div>
              <div>
                <div style={{ fontWeight: 600, fontSize: '17px', color: 'fff' }}>{platform.name}</div>
                {status?.valid && <div style={{ fontSize: '12px', color: '#4CAF50' }}>Validado @ {status.username || 'OK'}</div>
              </div>
            </div>
          </div>
        </div>
      )
    })}
  </div>
)

```

```

    </div>
    <button onClick={() => validatePlatform(platform.id)} disabled={status?.loading}
      style={{ background: status?.valid ? '#1B5E20' : '#1a1a2e', border: '1px solid ' + (status?.valid ? '#4CAF50' : '#3
33'),
        color: status?.valid ? '#4CAF50' : '#4FC3F7', padding: '8px 14px', borderRadius: '8px', fontSize: '13px', cursor:
'pointer', fontWeight: 600 }}>
      {status?.loading ? '...' : status?.valid ? 'OK' : 'Validar'}
    </button>
  </div>
  {platform.fields.map((field: any) => (
    <div key={field.label} style={{ marginBottom: '12px' }}>
      <label style={{ display: 'block', fontSize: '12px', color: '#666', marginBottom: '6px', fontWeight: 500, textTransf
orm: 'uppercase' }}>{field.label}</label>
      <textarea value={field.value} onChange={e => field.set(e.target.value)} placeholder={field.placeholder} rows={fiel
d.big ? 3 : 2}
        style={{ width: '100%', background: '#0a0a0a', border: '1px solid #222', borderRadius: '10px', padding: '12px', c
olor: '#fff', fontSize: '14px', fontFamily: 'monospace', resize: 'vertical', outline: 'none', boxSizing: 'border-box' }} />
      </div>
    ))}
  {status?.valid && <button onClick={() => testDm(platform.id)} style={{ width: '100%', background: platform.color + '2
2', border: '1px solid ' + platform.color + '44', color: platform.color, padding: '10px', borderRadius: '10px', fontSize: '13px', c
ursor: 'pointer', fontWeight: 600, marginTop: '4px' }}>Enviar DM de teste ({platform.name})</button>}
  </div>
);
}}
<div style={{ position: 'fixed', bottom: 0, left: 0, right: 0, padding: '16px', background: 'linear-gradient(transparent, #0a
0a0a 30%)' }}>
  <button onClick={saveCookies} style={{ width: '100%', padding: '16px', borderRadius: '14px', background: saved ? '#1B5E20'
: 'linear-gradient(135deg, #4FC3F7, #0288D1)', color: '#fff', fontSize: '16px', fontWeight: 700, border: 'none', cursor: 'pointer'
}}>
    {saved ? 'Cookies Guardados!' : 'Guardar Cookies'}
  </button>
</div>
</div>
);
}

```

MBA Brain Agent - Código Fonte Completo

7 ficheiros | Next.js + TypeScript | DMs via HTTP cookies